

GeoTools Package Manual

Caio Ciardelli

Northwestern University

Under the supervision of Prof. Suzan van der Lee

January 2026

Contents

1	Introduction	2
2	Dependencies	2
3	Directory Structure	3
4	Compilation	5
5	Running the Program	5
6	Key Components and Usage	6
6.1	resample_1d_model.py (utils/resample_1d_model.py)	6
6.2	compute_rays.py (utils/compute_rays.py)	7
6.3	mesher.c (bin/mesher)	9
6.4	ttimes.c (bin/ttimes)	9
6.5	gravity.c (bin/gravity)	11
6.6	plot_mesh.bash (utils/plot_mesh.bash)	11
6.7	plot_gravity.bash (utils/plot_gravity.bash)	11
6.8	pvsm_generator.py (utils/pvsm_generator.py)	12
7	Examples	13
8	Future Features	13

1 Introduction

The GeoTools Package is a geosciences software package developed for computing seismic ray paths, travel times, and synthetic gravity disturbances in 3D Earth models, including surface topography. It is written in C, Python, and Bash, and is designed for high-performance computations using parallel processing (via OpenMPI). The package is currently under development and focuses on forward modeling capabilities. The package now includes the capability to incorporate Moho topography into the mesh, affecting both gravity calculations and travel time computations. For travel times, Moho topography is accounted for by locally deforming the ray path, which serves as an approximation until full 3D ray tracing is implemented. Future versions will include inversion features for 3D velocity and density distributions, 3D ray tracing, and Earth's ellipticity. This manual provides a comprehensive guide to installing, compiling, running, and understanding the package. It includes details on the directory structure, dependencies, key scripts and programs, input/output files, and usage examples. The program is licensed under the GNU General Public License (GPL) version 3 or later. See the LICENSE file for details.

2 Dependencies

The GeoTools Package requires the following software and libraries:

- **Bash:** For running shell scripts (e.g., `run_test_workflow.bash`, `plot_mesh.bash`, `plot_gravity.bash`).
- **GCC:** C compiler for building the C binaries (`mesher`, `gravity`, `ttimes`).
- **Python 3.7 or later:** For Python scripts (`compute_rays.py`, `resample_1d_model.py`, `pvsm_generator.py`).
- **Python libraries:**
 - NumPy>1.26.4: Data processing and arrays (used in `compute_rays.py`, `resample_1d_model.py`).
 - Matplotlib>3.7.2: Plotting (used in `resample_1d_model.py` and optionally in `compute_rays.py`).
 - PvPython>3.12.7: Python with Paraview bindings (required for `pvsm_generator.py`).
 - OpenMPI>5.0: OpenMPI for parallel execution (required for gravity field computation).
 - SeisTracer: Seismic ray tracing library (required for `compute_rays.py`).
 - argparse: Command-line parsing (standard library).
 - heapq, math: Standard libraries for algorithms and calculations.
 - os, shutil: Standard libraries for file handling.
- **GMT (Generic Mapping Tools):** For generating plots (used in `plot_mesh.bash` and `plot_gravity.bash`).
- **ParaView:** Installation with Python support for 3D visualization (`pvsm_generator.py` generates `.pvsm` state files for interactive rendering).

- **OpenMPI:** For parallel execution in gravity.c (uses MPI for distributed computing across multiple cores).
- **bc:** Basic calculator for arithmetic operations in plot_gravity.bash.

Ensure all dependencies are installed on your system. For Python libraries, use pip:

```
1 pip install numpy matplotlib paraview
```

Note: SeisTracer requires additional installation steps; refer to its documentation. The package assumes access to ETOPO topography data in extra/earth_relief_15m.topo for surface topography incorporation and CRUST1.0 Moho data in extra/crust1_moho.dat for Moho topography incorporation.

3 Directory Structure

The package is organized as follows:

```
├── docs
│   ├── Manual.pdf
│   └── Manual.tex
├── extra
│   ├── CET-D01A.cpt
│   ├── crust1_moho.dat
│   └── earth_relief_15m.topo
├── input
│   ├── receivers.dat
│   └── sources.dat
├── LICENSE
├── Makefile
├── mesh
│   ├── facets.bin
│   ├── Mesh_VpVsRho.vtk
│   ├── r.bin
│   ├── rho.bin
│   ├── vertices.bin
│   ├── vp.bin
│   └── vs.bin
├── models
│   ├── ak135f.model
│   ├── iasp91.model
│   ├── prem.model
│   ├── rem1d.model
│   └── stw105.model
├── README.md
├── run_test_workflow.bash
├── setup
│   └── constants.h
```

```
├── src
│   ├── gravity.c
│   ├── headers
│   │   ├── coordinates.h
│   │   ├── exmath.h
│   │   ├── io.h
│   │   ├── progress.h
│   │   ├── structs.h
│   │   └── tessellation.h
│   ├── mesher.c
│   ├── shared
│   │   ├── coordinates.c
│   │   ├── exmath.c
│   │   ├── io.c
│   │   ├── progress.c
│   │   └── tessellation.c
│   └── ttimes.c
├── utils
│   ├── compute_rays.py
│   ├── plot_gravity.bash
│   ├── plot_mesh.bash
│   ├── pvsm_generator.py
│   └── resample_1d_model.py
```

Key directories:

- **extra:** Contains topography data (ETOPO: earth_relief_15m.topo), Moho data (CRUST1.0: crust1_moho.dat), and color palettes (CET-D01A.cpt) for GMT plotting.
- **input:** Input files for receivers (receivers.dat: station ID, lat, lon, elevation) and sources (sources.dat: event ID, lat, lon, depth) used in ray tracing.
- **mesh:** Output directory for mesh-related binary files (e.g., facets.bin, vertices.bin, r.bin, rho.bin, vp.bin, vs.bin) and VTK files (Mesh_VpVsRho.vtk).
- **models:** Predefined 1D Earth models (e.g., iasp91.model, prem.model) in text format for resampling and meshing.
- **setup:** Configuration header (constants.h) for C programs, controlling flags like INCORPORATE_SURFACE_TOPOGRAPHY (default: true) and DEBUG_MESH (default: true).
- **src:** C source code, including main programs (mesher.c, gravity.c, ttimes.c), headers, and shared utilities for math, I/O, and tessellation.
- **utils:** Utility scripts in Python and Bash for model resampling, ray computation, plotting, and ParaView state generation.

4 Compilation

The package uses a Makefile in the root directory for compilation. To build the C binaries (mesher, gravity, ttimes):

```
1 make all
```

This compiles the source files in src/ (using headers from src/headers and shared code from src/shared) and places the executables in a new bin/ directory. The compilation requires GCC and OpenMPI for parallel support in gravity. To clean previous build artifacts:

```
1 make clean
```

Note: Ensure GCC and OpenMPI are in your PATH. The Makefile assumes standard includes; modify it if needed for custom library paths. After compilation, grant execution permissions to utility scripts if necessary (e.g., `chmod +x utils/*.bash`).

5 Running the Program

The main entry point is the test workflow script: `run_test_workflow.bash`. It automates the entire forward modeling process: model resampling, ray path computation, mesh generation, travel time calculation, gravity computation, and visualizations. The script uses a fixed refinement level (7), model (IASP91), and parameters (e.g., NCORES=6, DELTA=5.0, HEIGHT=255 km), but these can be customized by editing the script.

Usage:

```
1 ./run_test_workflow.bash
```

Steps performed by the script:

- Cleans previous outputs (removes phases directories, .txt, .dat, .png, .vtk, .pvsm, gmt.history).
- Resamples the 1D IASP91 model to input/input.model using `resample_1d_model.py`.
- Computes ray paths for sample source/receiver pairs using `compute_rays.py`, outputting to `phases_EQ001_STA01/`.
- Generates the mesh with refinement 7 using `mesher`.
- Plots the mesh with `plot_mesh.bash`, creating PNGs like `mesh320.png` and `nodes_spiral.png`.
- Generates a ParaView state file (e.g., `Mesh_VpVsRho.pvsm`) with `pvsm_generator.py` for mesh and selected phases (PKIIKP, etc.).
- Computes travel times for phases in `phases_EQ001_STA01/` using `ttimes`, outputting to `ttimes.txt`.

- Computes synthetic gravity in parallel (mpirun -n 6) using gravity with DELTA=5.0 and HEIGHT=255, outputting G_*.dat files. Note: Requires 16 GB RAM; reduce NCORES, layers, or refinement if memory is limited.
- Plots gravity with plot_gravity.bash on G_normal.dat, creating G_normal.png.

For help:

```
1 ./run_test_workflow.bash --help
```

Individual components can be run manually (see Section 6). Outputs are placed in the current directory or subdirectories like mesh/ and phases_EQ001_STA01/.

6 Key Components and Usage

6.1 resample_1d_model.py (utils/resample_1d_model.py)

This Python script resamples a 1D Earth model in two stages: linear collapse (removes points in constant-gradient intervals) and importance-based pruning (removes least important points until target ratio). It preserves header lines and outputs stats, errors, and an optional comparison plot. Usage:

```
1 python utils/resample_1d_model.py -i INPUT -o OUTPUT [options]
```

Key options (from argparse):

- -i, -input: Input model file (required, e.g., models/iasp91.model).
- -o, -output: Output resampled model file (required).
- -depth-col: Depth column index (default: 0).
- -check-cols: Columns to resample (default: all except depth).
- -rtol: Relative slope tolerance for linear collapse (default: 1e-6).
- -atol: Absolute slope tolerance (default: 1e-8).
- -target-keep-ratio: Fraction of points to keep (default: 0.50).
- -normalize: Normalize errors by column range (default: True).
- -error-agg: Error aggregation method (default: max).
- -preserve-zero: Preserve points where any checked column is zero (default: True).
- -enable-linear: Enable linear collapse stage (default: True).
- -save-fig: Save comparison plot as OUTPUT.png.
- -max-cols: Max columns to plot (default: 4).

- `-header-lines`: Header lines to preserve (default: 4).
- `-fortran-d`: Handle Fortran D notation (default: True).

Input: Text file with header (4 lines) and data columns (e.g., depth, V_p , V_s , ρ). Output: Resampled model file, console stats (reduction %, max errors), optional PNG plot. Example (from workflow): `python utils/resample_1d_model.py -i models/iasp91.model -o input/input.model`

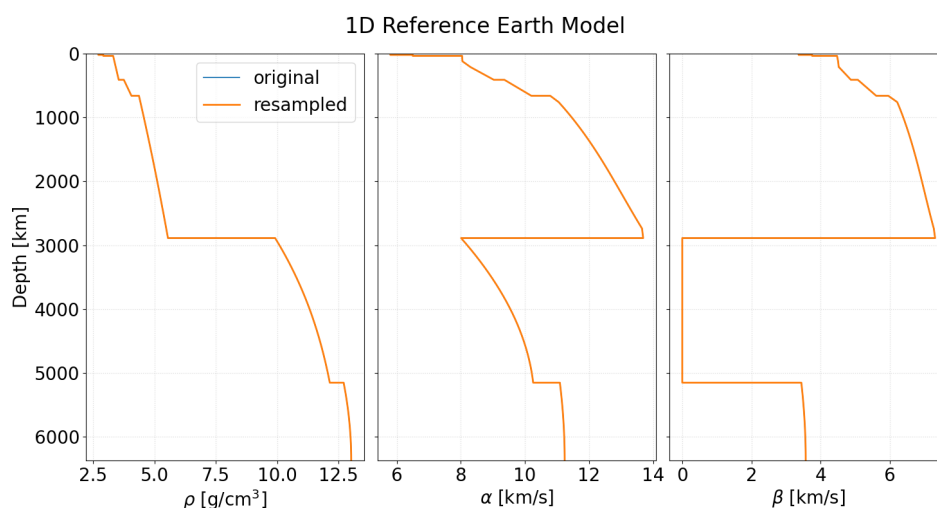


Figure 1: Comparison between the original and resampled 1D reference Earth model.

6.2 `compute_rays.py` (`utils/compute_rays.py`)

This Python script computes seismic ray paths for source-receiver pairs using SeisTracer. It generates directories with VTK/text files for rays, phase lists, and optional PNG plots. Supports multiple phases and saves receiver elevation. Usage:

```
python utils/compute_rays.py -r RECEIVERS -s SOURCES [options]
```

Key options:

- `-r`: Receivers file (required, e.g., `input/receivers.dat`: ID lat lon elev).
- `-s`: Sources file (required, e.g., `input/sources.dat`: ID lat lon depth).
- `-no-png`: Disable PNG ray path plots.
- `-no-vtk`: Disable VTK ray path files.

Input: `receivers.dat` and `sources.dat` (text files with locations). Output: Directory per pair (e.g., `phases_EQ001_STA01/`) with:

- `PHASE.vtk` / `PHASE.txt`: Ray coordinates and branches.
- `sr.vtk`: Source/receiver points.

- phases_list.txt: List of computed phases.
- receiver_elevation.txt: Receiver elevation in meters.
- Optional PHASE.png: Ray path plots.

Example (from workflow): `python utils/compute_rays.py -r input/receivers.dat -s input/sources.dat` Notes: Uses Vincenty's formula for great-circle distances; maps branch types (P=0, S=1).

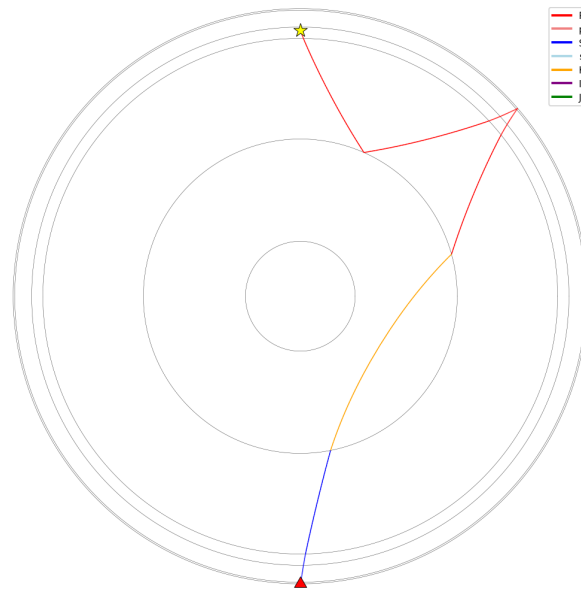


Figure 2: Example of a single seismic ray path computed for a complex phase.

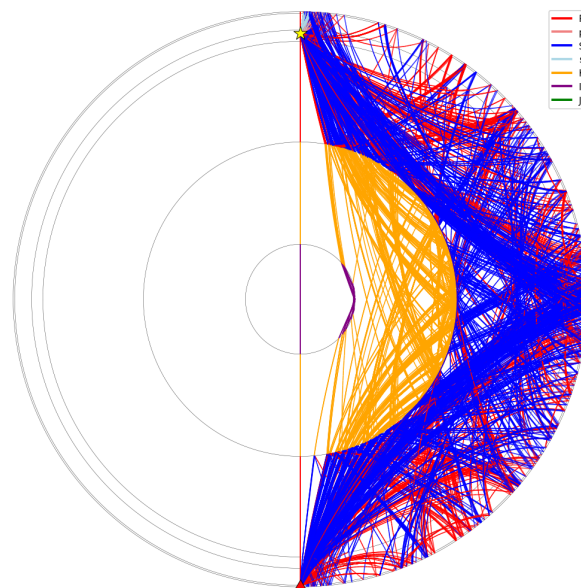


Figure 3: Multiple seismic ray paths for various phases between a source and receiver pair.

6.3 mesher.c (bin/mesher)

This C program creates a spherical mesh based on a 1D velocity model, optionally incorporating ETOPO topography (with Gaussian anti-aliasing filter), CRUST1.0 Moho topography (similarly filtered), and 3D perturbations. Starts from an icosahedron and refines it. The Moho topography is incorporated by adjusting the radii at the Moho discontinuity level, using a similar anti-aliasing filter as for surface topography. By default, it uses the Moho depths from the CRUST1.0 model in `extra/crust1_moho.dat`, but users can specify a different file by changing the variable `MOHO_FILE_NAME` in `constants.h`.

Usage:

```
1 ./bin/mesher REFINEMENT MODEL
```

- **REFINEMENT**: Number of refinement iterations (e.g., 7; increases vertices/facets exponentially).
- **MODEL**: Path to 1D model file (e.g., `input/input.model`).

Output (in `mesh/` and `current` dir):

- **Binary**: `facets.bin` (facet data), `vertices.bin` (vertex data), `r.bin/rho.bin/vp.bin/vs.bin` (model values per vertex/shell).
- **Text**: `nodes.txt` (lon/lat of vertices), `spiral.txt` (spiral-ordered vertices), `mesh_edges_%d.txt`, `mesh_triangles_%d.txt`, `mesh_center_triangle_%d.txt` (for `DEBUG_MESH=true`).
- **VTK**: `Mesh_VpVsRho.vtk` (3D mesh with `Vp`, `Vs`, `Rho` scalars).

Configurable in `constants.h`:

- `INCORPORATE_SURFACE_TOPOGRAPHY`: `true` (uses `extra/earth_relief_15m.topo`).
- `INCORPORATE_MOHO_TOPOGRAPHY`: `true` (uses `extra/crust1_moho.dat`).
- `INCORPORATE_3D_MODEL`: `false` (placeholder for 3D perturbations).
- `DEBUG_MESH`: `true` (outputs text files for plotting).

Example (from workflow): `./bin/mesher 7 input/input.model` Notes: Prints mesh stats (facets, vertices, edge lengths). Requires number of shells from model.

6.4 ttimes.c (bin/ttimes)

This C program computes seismic travel times for phases using the mesh and model, accounting for topography via receiver elevation. When Moho topography is incorporated (via `INCORPORATE_MOHO_TOPOGRAPHY` in `constants.h`), it is accounted for by locally deforming the ray path near the Moho discontinuity. This deformation adjusts the ray to follow the varying Moho depth, providing an approximation to the true 3D path perturbation caused by the Moho topography. This is an interim solution; the ideal approach of full 3D ray tracing will replace this approximation in future versions.

Usage:

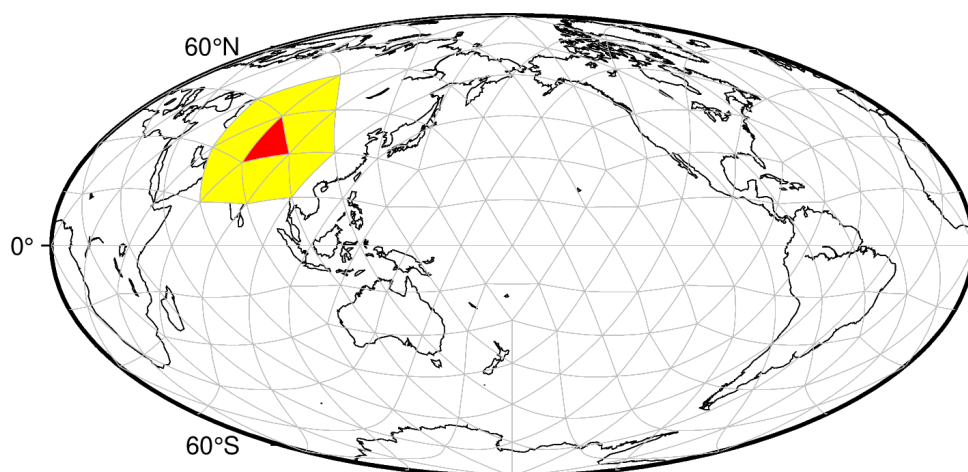


Figure 4: Global map showing a highlighted triangle in the spherical mesh.

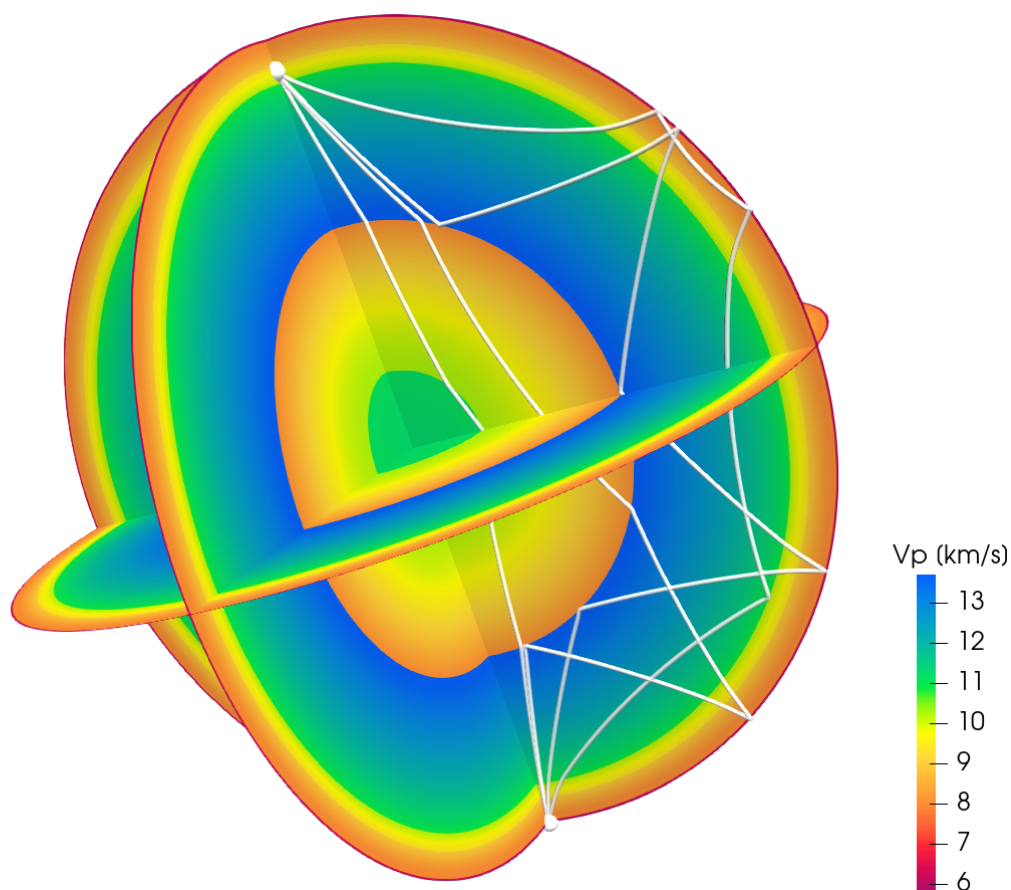


Figure 5: 3D visualization of the mesh colored by P-wave velocity (V_p).

```
1 ./bin/ttimes INPUT_DIR > ttimes.txt
```

- INPUT_DIR: Directory with phases_list.txt (phase names) and PHASE.txt (ray coordinates/branches), receiver_elevation.txt.

Output: To stdout: Phase names and travel times (seconds). Example (from workflow): `./bin/ttimes phases_EQ001_STA01/ > ttimes.txt` Notes: Reads mesh/model binaries; resamples rays with DELTA=0.1 km; uses barycentric coordinates for element location.

6.5 gravity.c (bin/gravity)

This C program computes Earth's gravitational field in parallel using MPI, accounting for topography and height. When Moho topography is incorporated into the mesh (via INCORPORATE_MOHO_TOPOGRAPHY in constants.h), it affects the gravity calculations by altering the density distribution at the Moho level in the 3D model. Usage:

```
1 mpiexec -n NCORES ./bin/gravity DH HEIGHT
```

- DH: Grid spacing in degrees (e.g., 5.0).
- HEIGHT: Observation height in km (e.g., 255).

Output: G_x.dat, G_y.dat, G_z.dat (gravity components), G_normal.dat (normal component, in g [m/s²]). Notes: Reads mesh/model binaries; decomposes elements into tetrahedra; computes mass (prints Earth's mass). Parallel over facets. Example (from workflow): `mpiexec -n 6 ./bin/gravity 5.0 255`

6.6 plot_mesh.bash (utils/plot_mesh.bash)

This Bash script generates GMT plots of mesh triangles, centers, and edges from text files.

Usage:

```
1 ./utils/plot_mesh.bash
```

Input: Auto-detects mesh_*.txt files in current directory. Output: PNGs like mesh%d.png (triangles in yellow, centers in red, edges in gray if <=20480), nodes_spiral.png (if max=320: edges, spiral in magenta, nodes in green). Example (from workflow): `./utils/plot_mesh.bash`

6.7 plot_gravity.bash (utils/plot_gravity.bash)

This Bash script generates a GMT plot of gravity data (e.g., normal component).

Usage:

```
1 ./utils/plot_gravity.bash FILENAME
```

- FILENAME: Base name without .dat (e.g., G_normal).

Input: FILENAME.dat (lon lat gravity). Output: FILENAME.png (global map with colorbar in mGal, using extra/CET-D01A.cpt). Example (from workflow): `./utils/plot_gravity.bash G_normal` Notes: Grids data with xyz2grd/surface; rescales to mGal; avoids poles.

Synthetic Gravity - ETOPO

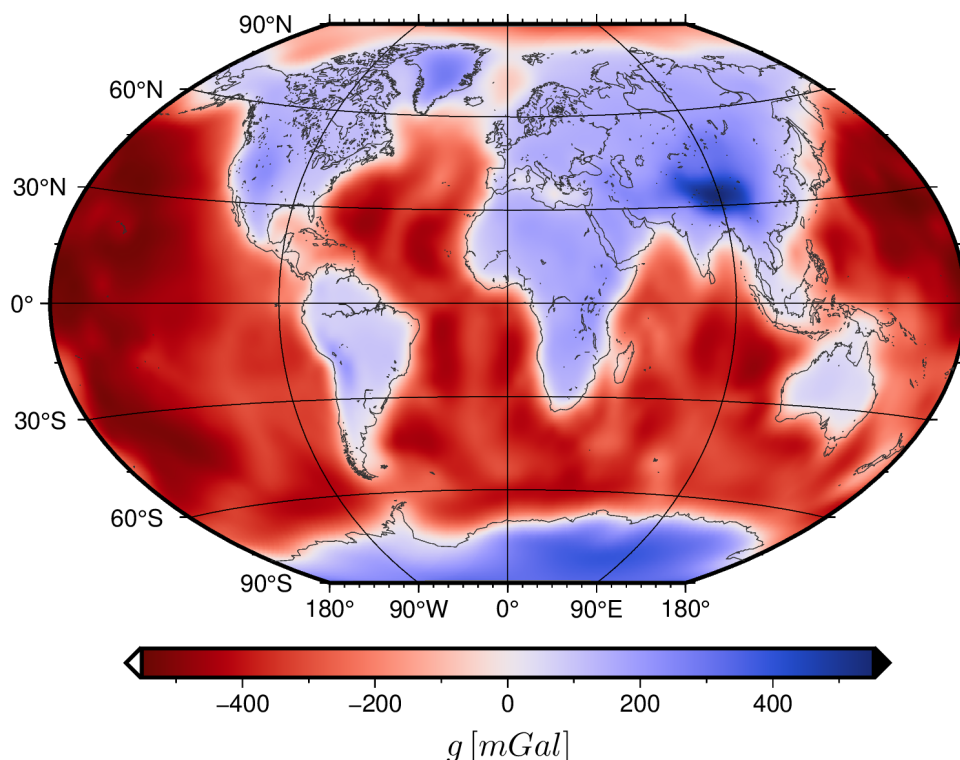


Figure 6: Synthetic gravity map incorporating ETOPO topography (Moho topography was set to 'false' in this example).

6.8 pvsm_generator.py (utils/pvsm_generator.py)

This Python script generates ParaView .pvsm state files for VTK mesh/ray visualization. Usage:

```
1 pvpython utils/pvsm_generator.py MESH_VTK [options]
```

Key options:

- `-color-by`: Scalar to color by (e.g., `Vp`).
- `-phases-dir`: Directory with ray VTKs.
- `-phases`: Space-separated phases (e.g., `P S`).
- `-reverse-colormap`: Reverse color map.
- `-camera-view`: View angles (e.g., `210,10,0,0,20`).
- `-light-intensity/azimuth/elevation`: Light settings.
- `-label`: Colorbar label (e.g., `'Vp [km/s]'`).

Output: `MESH_VTK.pvsm` (ParaView state file). Example (from workflow): `pvpython utils/pvsm_generator.py mesh/Mesh_VpVsRho.vtk -color-by Vp -phases-dir phases_EQ001_STA01`

–phases PKIIKP PKPPm+PPcS ScSScSm-ScS SPS660-S –reverse-colormap –camera-view 210,10,0,0,20 –label 'Vp [km/s]' –light-intensity 0.5 –light-azimuth 140 –light-elevation 35 Notes: Requires pypython or ParaView Python env; adds rays as tubes, sources/receivers as spheres.

7 Examples

Run the full test workflow:

```
1 ./run_test_workflow.bash
```

This uses sample input/sources.dat (one event) and input/receivers.dat (one station), processes IASP91, and generates all outputs. Customize rays: Edit input/sources.dat and receivers.dat for multiple events/stations. View in ParaView: Load Mesh_VpVsRho.pvsm for interactive 3D mesh/rays. Run mesher standalone: ./bin/mesher 8 models/prem.model Plot custom gravity: ./utils/plot_gravity.bash my_gravity

8 Future Features

- 3D ray tracing in ttimes.
- Earth's ellipticity corrections.
- Inversion capabilities for 3D velocity/density and Moho topography.

For contributions or issues, contact the author.